

A bit of theory as far as relevant to the implementation of `class LagrangeMesh`

The necessary theory - for non-reduced axes - is found in [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation. The corresponding section in [Baye, Phys. Rep. 565, 1 \(2015\)](#) is 3.7.1. Lagrange–Fourier mesh.

Note

The figures in this document are created by the tests. If the document complains `blablablah.png could not be found.`, please run the tests (`pytest tests` from folder `MOCCaPy`)

Non-reduced axes

(As from [Ryssens et al, PHYSICAL REVIEW C 92, 064318 \(2015\)](#) section III.B Lagrange-mesh representation)

Grid points

Using M evenly space grid point at distance Δ , the boundaries of the box are $[-\frac{M}{2}\Delta, \frac{M}{2}\Delta]$. The width of the box is $L = M\Delta$. We require $M = 2N$ to be even such that there are $N = \frac{M}{2}$ grid points on each side of the origin and the origin is avoided as a grid point.

For non-reduced axes (where the shift parameter σ is required be zero) the grid points are:
[eq 1]

$$\pm \frac{1}{2}\Delta, \pm \frac{3}{2}\Delta, \dots, \pm \frac{2N-3}{2}\Delta, \pm \frac{2N-1}{2}\Delta$$

or

[eq 2]

$$x_{\pm i} = \pm \left(\frac{1}{2} + i\right)\Delta = \pm \left(\frac{2i+1}{2}\right)\Delta$$

$$i = 0..N-1$$

(the notation $x_{\pm i}$ is not entirely accurate as it must distinguish between $+0$ and -0 .)

In increasing order:

[eq 3]

$$-\frac{2N-1}{2}\Delta, -\frac{2N-3}{2}\Delta, \dots, -\frac{3}{2}\Delta, -\frac{1}{2}\Delta, \frac{1}{2}\Delta, \frac{3}{2}\Delta, \dots, \frac{2N-3}{2}\Delta, \frac{2N-1}{2}\Delta$$

or

[eq 4]

$$x_i = -\frac{2N-1-2i}{2}\Delta = \left(\frac{1}{2} + i - N\right)\Delta$$
$$i = 0..(2N-1)$$

or

[eq 4.1]

$$x_i = -\frac{2N-1-2i}{2}dx = \left(\frac{1}{2} + i - N\right)dx$$
$$i = -(N-1)..(N-1)$$

Basis functions

The basis functions are plane waves:

[eq 5]

$$\phi_k(x) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L} kx\right)$$

where we choose j for the imaginary unit (rather than j , as j and j are a bit better distinguishable than j and i), and

$$k = \pm\frac{1}{2}, \pm\frac{3}{2}, \dots, \pm\frac{2N-3}{2}, \pm\frac{2N-1}{2}$$

or, since $k\Delta$ is a grid point, say, the i -th, x_i :

[eq 5]

$$\phi_{x_i}(x) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L} \frac{x_i}{\Delta} x\right) = \frac{1}{\sqrt{L}} \exp\left(\frac{2\pi j}{L\Delta} x_i x\right)$$

Note

Since $\exp(jz) = \cos z + j \sin z$ the real component of the 1D basis function is symmetric and the imaginary component is skew-symmetric. Consequently, we can use them for testing interpolation with Lagrange functions (see below) also on reduced grids.

$$\text{Re}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \cos\left(\frac{2\pi}{L\Delta} x_i x\right)$$

$$\text{Im}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \sin\left(\frac{2\pi}{L\Delta} x_i x\right)$$

First order derivatives are:

$$\frac{d}{dx} \text{Re}(\phi_{x_i}(x)) = -\frac{1}{\sqrt{L}} \frac{2\pi}{L\Delta} \sin\left(\frac{2\pi}{L\Delta} x_i x\right)$$

$$\text{Im}(\phi_{x_i}(x)) = \frac{1}{\sqrt{L}} \frac{2\pi}{L\Delta} \cos\left(\frac{2\pi}{L\Delta} x_i x\right)$$

Interpolation

The Lagrange interpolation functions are:

[eq 6]

$$f_i(x) = \frac{1}{2N} \frac{\sin\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)}$$

By construction, the Lagrange interpolation functions $f_i(x)$ have the property of being equal to 1 at the i -th mesh point, x_i , and 0 at all others:

[eq 7]

$$f_i(x_j) = \delta_{ij}$$

Note that when evaluating $f_i(x_i)$, both the numerator and the denominator are 0, and Python yields a `nan`. We need to invoke l'Hôpital's rule to obtain the result.

[eq 7.1]

$$\begin{aligned} & \lim_{x \rightarrow x_i} \frac{1}{2N} \frac{\sin\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\sin'\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\sin'\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{1}{2N} \lim_{x \rightarrow x_i} \frac{\frac{\pi}{\Delta} \cos\left(\frac{\pi}{\Delta}(x - x_i)\right)}{\frac{\pi}{2N\Delta} \cos\left(\frac{\pi}{\Delta} \frac{x - x_i}{2N}\right)} \\ &= \frac{2N}{2N} \lim_{x \rightarrow x_i} \frac{\cos(0)}{\cos(0)} = 1 \end{aligned}$$

In order to avoid testing for $\frac{0}{0}$ it suffices to add $\varepsilon \approx 1e - 9$ to x_i to avoid the `nan` and yield something close to 1.

The arguments of the two sine functions in eq 6 are the same, apart from a factor $1/2N$:

[eq 8]

$$\begin{aligned} f_i(x) &= \frac{1}{2N} \frac{\sin(A_i(x))}{\sin\left(\frac{A_i(x)}{2N}\right)} \\ A_i(x) &= \frac{\pi}{\Delta}(x - x_i) \end{aligned}$$

This can be exploited for efficiency in a computation.

An arbitrary function $h(x)$ taking the values $h_i = h(x_i)$ on the grid points can be interpolated on an arbitrary point $x \in [-N\Delta, N\Delta]$ as:

[eq 9]

$$h(x) = \sum_{i=0}^{2N-1} h(x_i) f_i(x)$$

This is essentially a dot product $\mathbf{f} \cdot \mathbf{h}$.

Derivatives

The 1st and 2nd order derivatives of the Lagrange interpolation functions are:

[eq 10.1]

$$D_{ji}^{(1)} = \left. \frac{df_i(x)}{dx} \right|_{x=x_j} = \begin{cases} (-1)^{i-j} \frac{\pi}{2N\Delta} \frac{1}{\sin(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ 0, & \text{for } i = j. \end{cases}$$

[eq 10.2]

$$D_{ji}^{(2)} = \left. \frac{d^2 f_i(x)}{dx^2} \right|_{x=x_j} = \begin{cases} (-1)^{i-j+1} 2 \left(\frac{\pi}{2N\Delta} \right)^2 \frac{\cos(\pi(i-j)/2N)}{\sin^2(\pi(i-j)/2N)}, & \text{for } i \neq j. \\ -\frac{\pi^2}{3\Delta^2} \left(1 - \frac{1}{(2N)^2} \right), & \text{for } i = j. \end{cases}$$

⚠ Warning

I suspect that there is a subtle problem with these formulas. When coding them the unit test failed giving the right values but the opposite sign. While attempting to derive the formulas i discovered that using the definition in eq 6, taking derivatives and evaluating in x_j leads to $(x_j - x_i)$ in the arguments of the sine functions, which is equal to $(j - i)\Delta$, and NOT $(i - j)$ as written in the formulas. The derivation proceeds by considering separate cases for odd and even $(j - i)$ and noting that $\cos \pi(2n)$ and $\sin \pi(2n + 1)$ vanish, yielding the sign factor in eq 10.1 and 10.2., $(-1)^{(i-j)} = (-1)^{(j-i)}$, but obviously the change from $(i - j)$ to $(j - i)$ in the sign functions yields a sign flip. (suspicion confirmed)

By applying these to the expansion $\phi(x) = \sum_{i=0}^{2N-1} \phi(x_i) f_i(x)$ we obtain

[eq 11]

$$\frac{d\mathbf{h}}{dx} = \left. \frac{dh(x)}{dx} \right|_{x=x_j} = \sum_{i=0}^{2N-1} \left. \frac{df_i(x)}{dx} \right|_{x=x_j} h(x_i) = \sum_{i=0}^{2N-1} D_{ji}^{(1)} h(x_i) = \mathbf{D}^{(1)} \mathbf{h}$$

So, the column vector $\frac{d\mathbf{h}}{dx}$ of the derivatives of $\mathbf{h}$ at all grid points is found as a matrix product of $\mathbf{D}^{(1)}$ with the column vector of the values of $\mathbf{h}$ at all the gridpoints. As the 2nd derivative forms a matrix as well, we also have

[eq 12]

$$\frac{d^2 \mathbf{h}}{dx^2} = \mathbf{D}^{(2)} \mathbf{h}$$

Note, that single differentiation toggles the symmetry behavior of the function: if $h(x)$ is a

symmetric function, $h'(x)$ is skew-symmetric, and *vice versa*. Consequentially, double differentiation toggles it twice, hence keeps the symmetry behaviour the same.

Reduced axes

Grid points

On reduced axes only the N grid points to the right of the origin are kept:

[eq 13]

$$\frac{1}{2}dx, \frac{3}{2}dx, \dots, \frac{2N-3}{2}dx, \frac{2N-1}{2}dx$$

or

[eq 14]

$$x_i = (i + \frac{1}{2})dx$$

$$i = 0..N-1$$

(In the case of reduced axis a shift $\sigma < dx$ may be subtracted from each grid point.)

Interpolation

According to (eq 9) an arbitrary function $h(x)$ can be interpolated as:

$$h(x) = \sum_{i=0}^{2N-1} h(x_i) f_i(x)$$

If the x -axis is reduced, i.e. we require $h(x)$ to be symmetric ($h(-x) = h(x)$) or skew-symmetric ($h(-x) = -h(x)$), then the above equation becomes:

$$h(x) = \sum_{i=0}^{N-1} h(x_i) f_i(x) + \sum_{i=0}^{N-1} h(-x_i) f_{-i}(x)$$

where $f_{-i}(x)$ is the Lagrange interpolation function corresponding to the i -th grid point to the left of the origin

[eq 14.1]

$$f_{-i}(x) = \frac{1}{2N} \frac{\sin(\frac{\pi}{\Delta}(x - x_{-i}))}{\sin(\frac{\pi}{\Delta} \frac{x - x_{-i}}{2N})} = \frac{1}{2N} \frac{\sin(\frac{\pi}{\Delta}(x + x_i))}{\sin(\frac{\pi}{\Delta} \frac{x + x_i}{2N})}$$

(where again we must distinguish a bit sloppily between $i = +0$ and $i = -0$ as the first mesh point index right, resp. left of the origin).

This then becomes:

$$h(x) = \sum_{i=0}^{N-1} \begin{cases} h(x_i)(f_i(x) + f_{-i}(x)), & h \text{ is symmetric} \\ h(x_i)(f_i(x) - f_{-i}(x)), & h \text{ is skew-symmetric} \end{cases}$$

or

[eq 15]

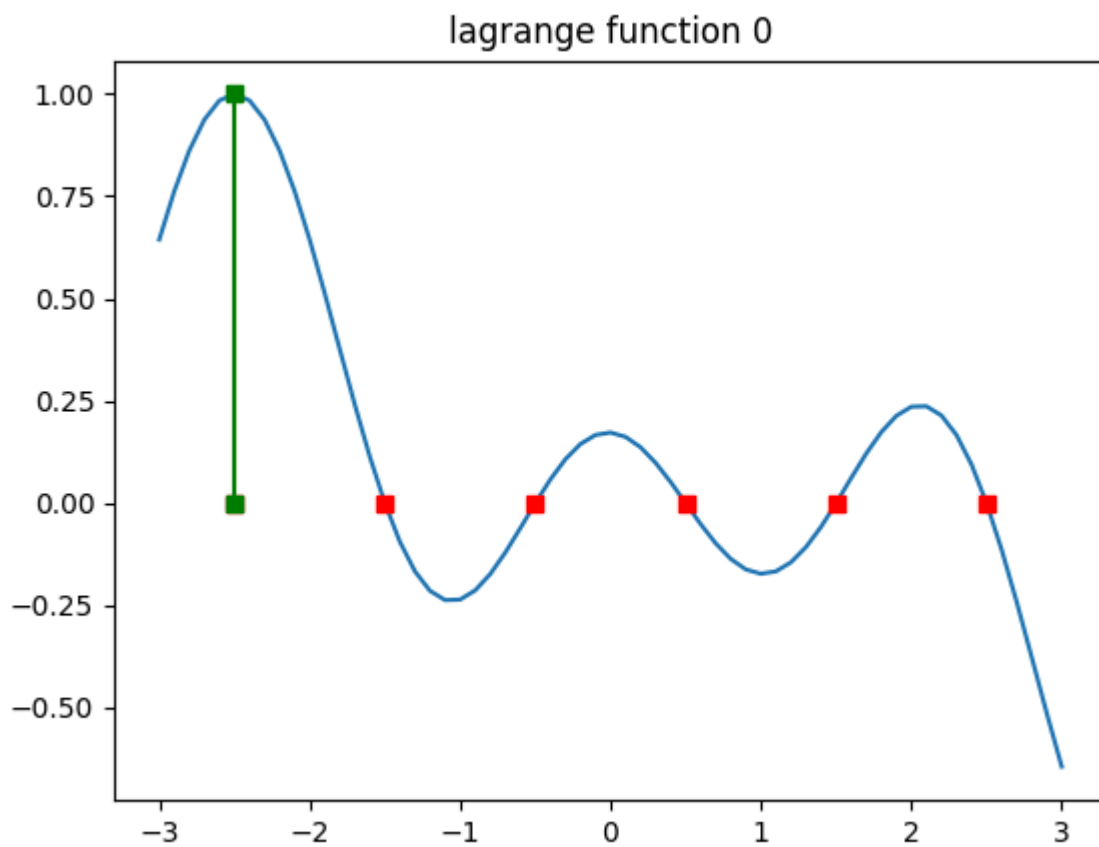
$$h(x) = \sum_{i=0}^{N-1} h(x_i)(f_i(x) \pm f_{-i}(x)) = \mathbf{h} \cdot \mathbf{f}_{\pm}$$

where the $\pm$ is $+$ for the symmetric case and $-$ for the skew-symmetric case, and $\mathbf{f}_{\pm} = \mathbf{f}_+ \pm \mathbf{f}_-$

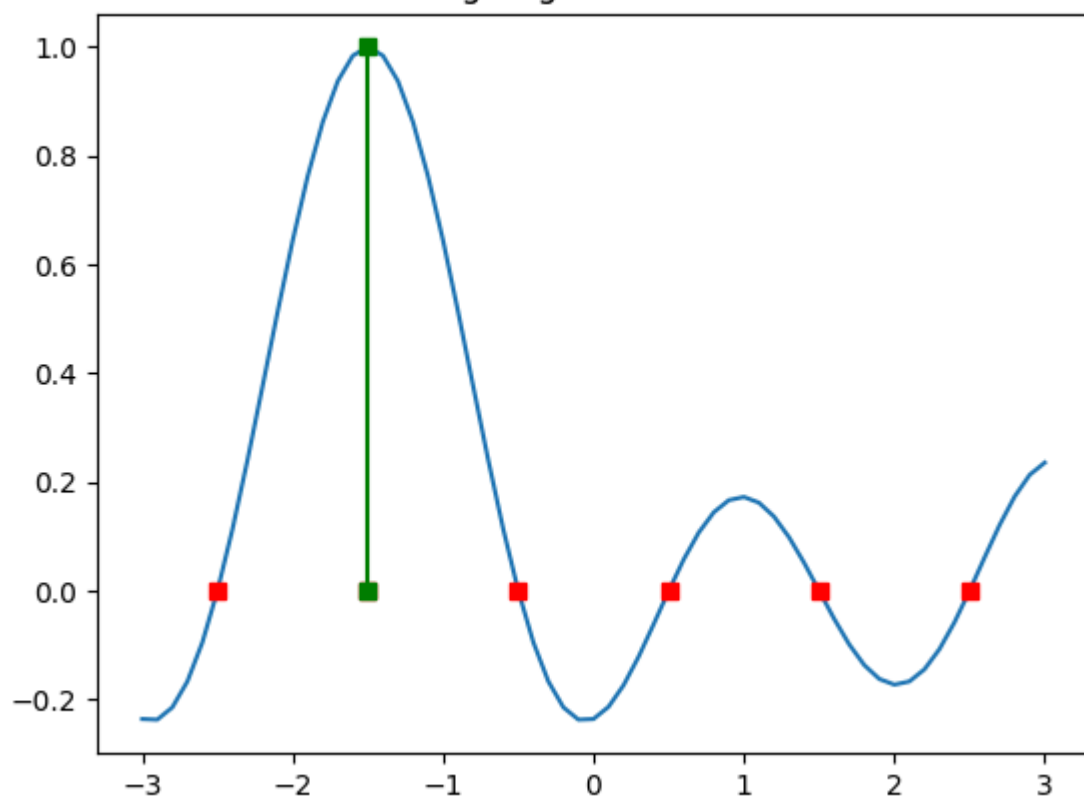
.

General remark on interpolation with Lagrange functions

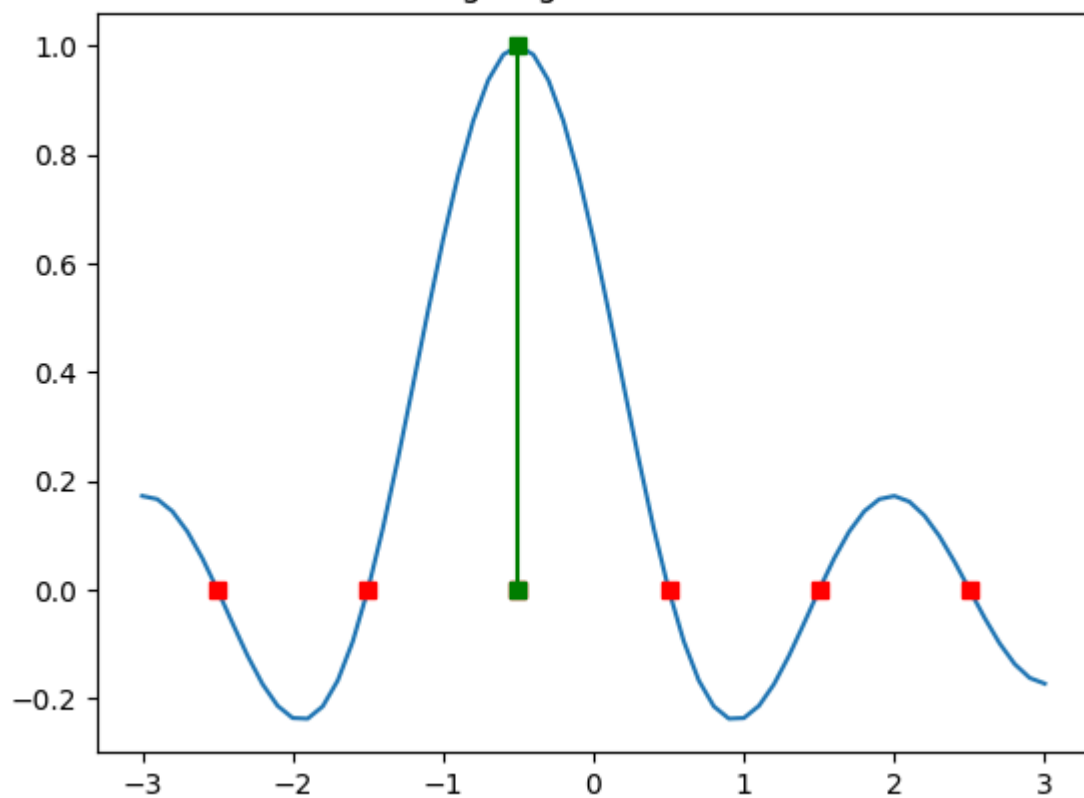
On a 1D grid with 6 grid points at $-1.25, -.75, -.25, .25, .75, 1.25$ on the interval $[-1.5, 1.5]$, these are the 6 Lagrange functions:



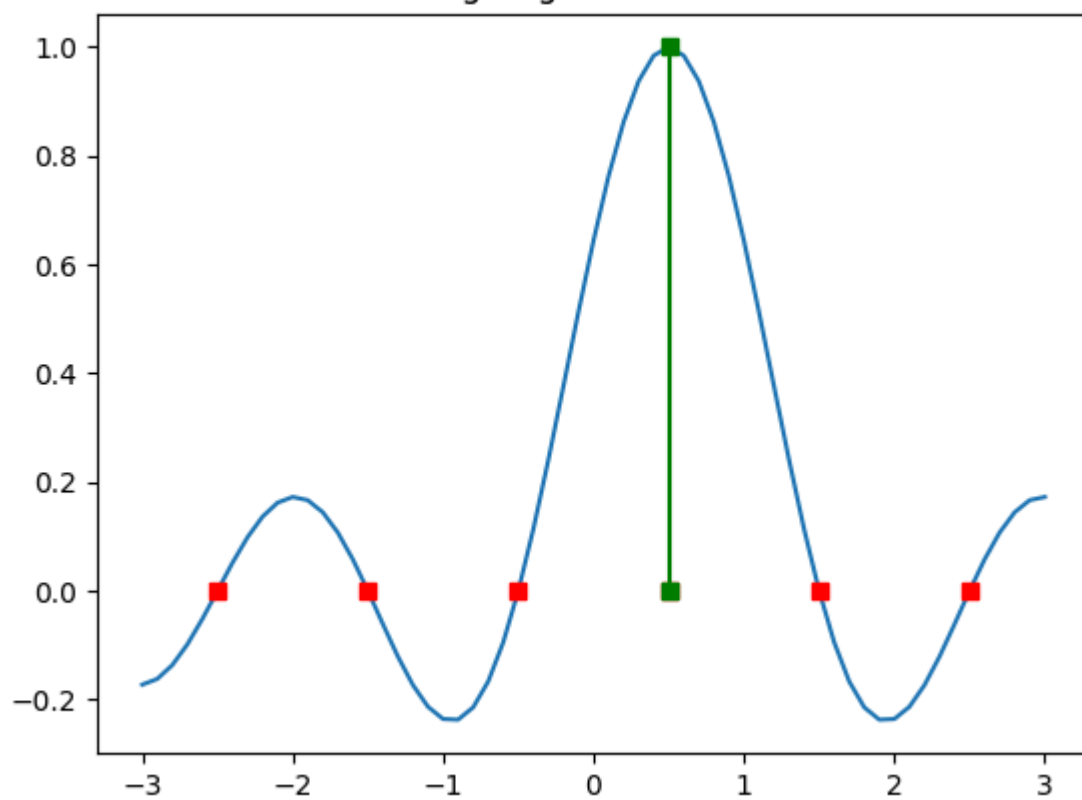
lagrange function 1



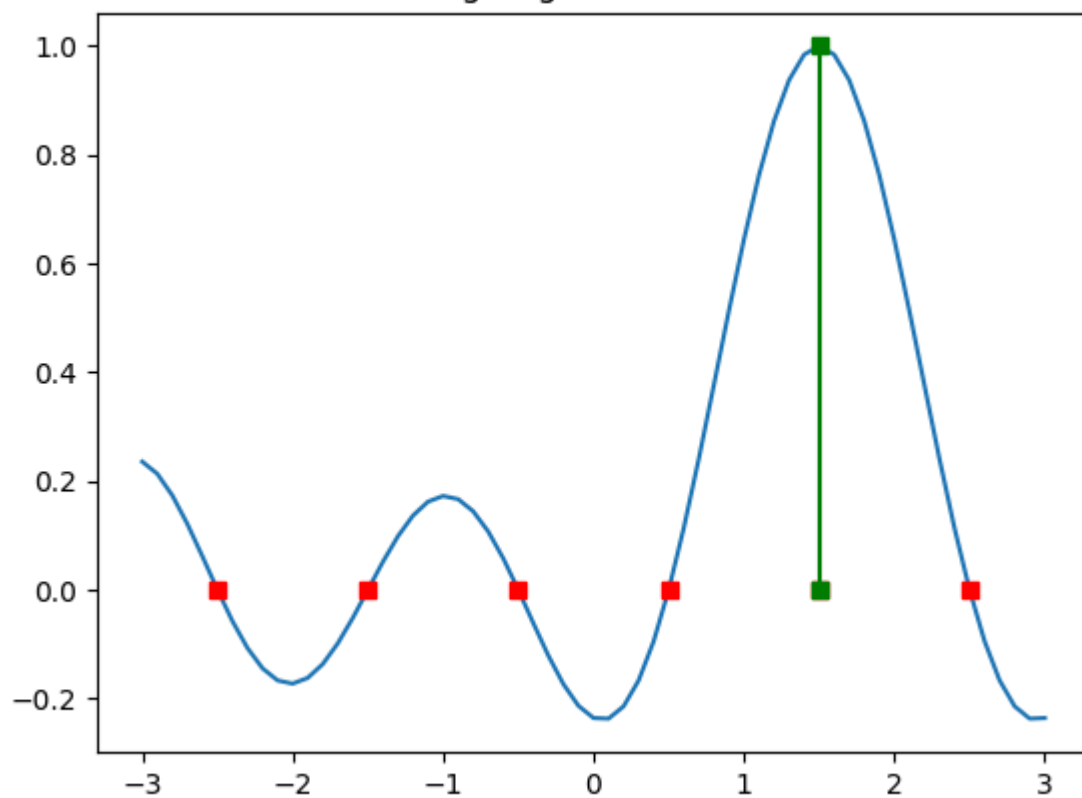
lagrange function 2

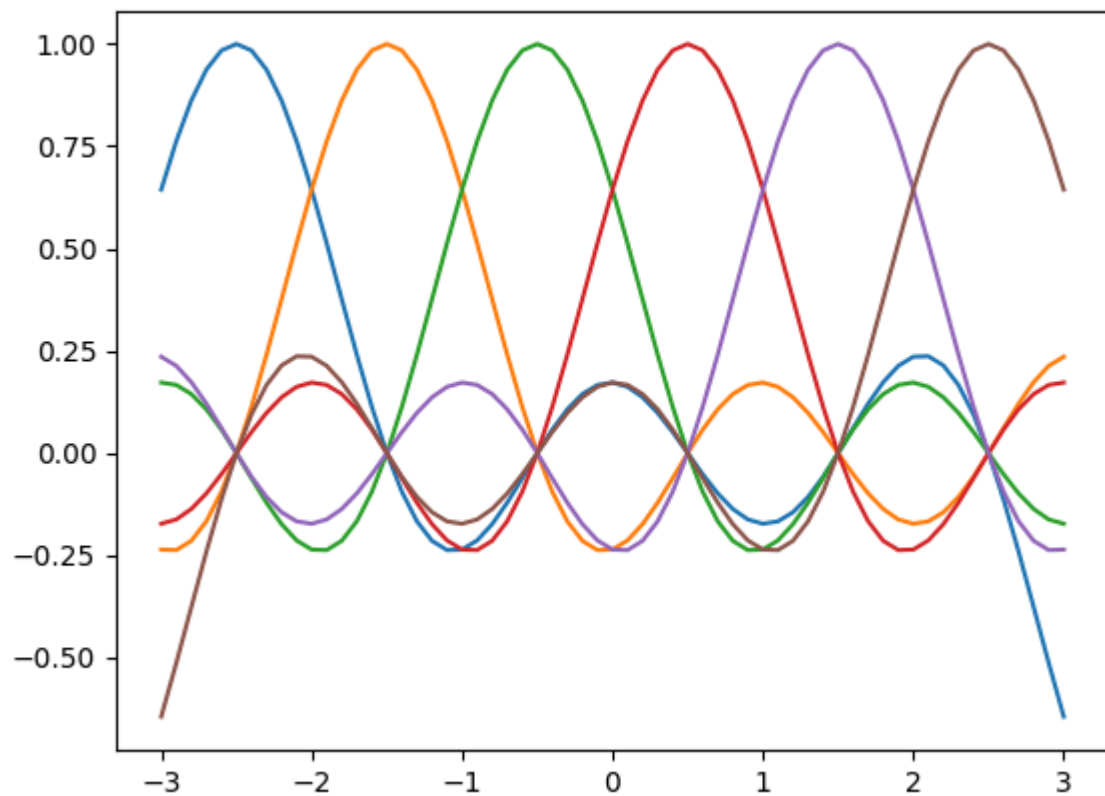
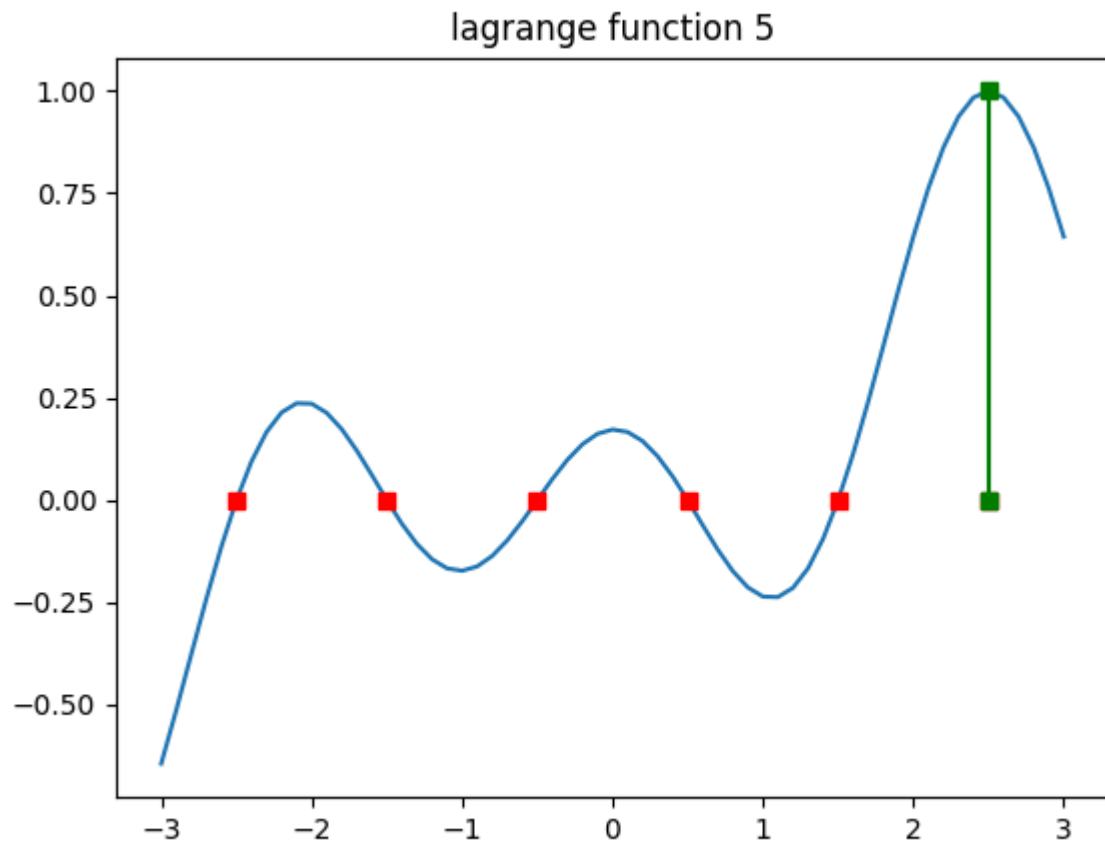


lagrange function 3



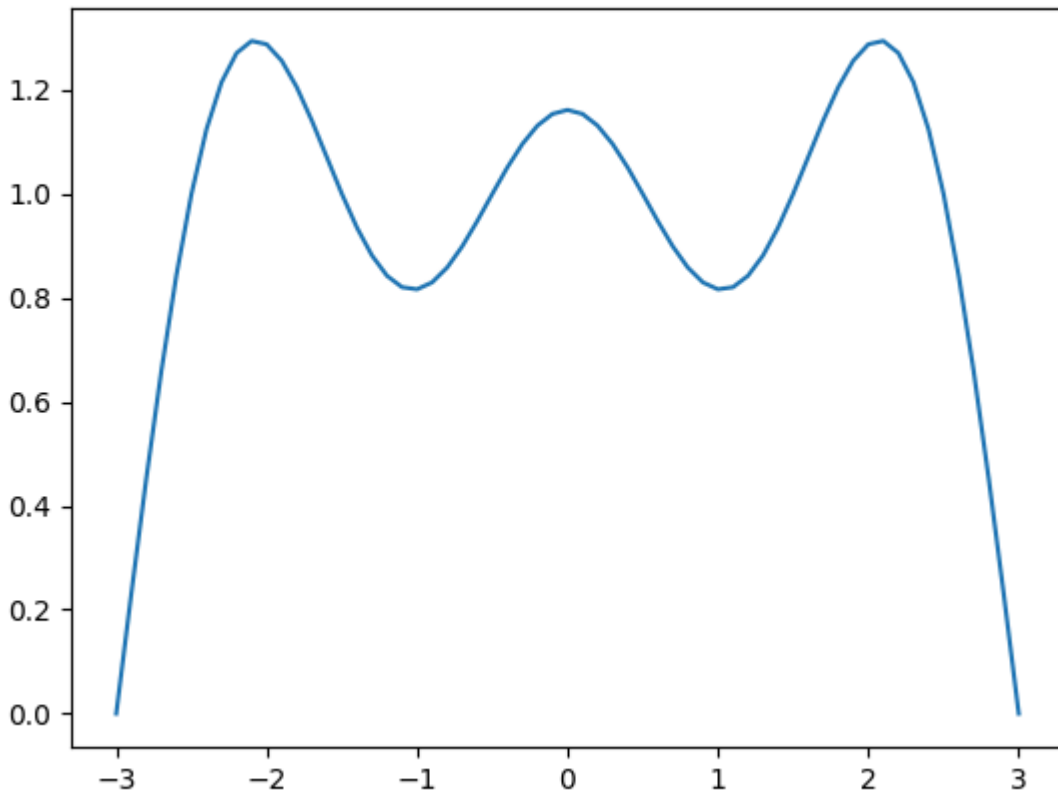
lagrange function 4





Clearly, each one yields 1 at one grid point and 0 at the others. Note also that they are antiperiodic: they reenter the box at the opposite edge, but **with a sign change**. Consequently, a constant function cannot be interpolated because it is periodic. The sum of

the 6 Lagrange functions is shown below. it is definitely not the constant function $f(x) = 1$.



According to the discussion in [github issue 52](#) periodic functions can be interpolated with Lagrange functions provided they vanish at the boundary of the interval.

Derivatives

The formula for the derivative of a function h expanded on a reduced grid is found easily by extending the column vector $\mathbf{h}$ (of length N) on the reduced grid as
[eq 17]

$$\begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix}$$

with $\mathbf{g}$ containing the same elements as $\mathbf{h}$ but in reverse order, as if they were mirrored by a horizontal line between the two. The full vector then corresponds to the $\mathbf{h}_{\text{non-reduced}}$ case.

This allows for the matrix product :

[eq 18]

$$\mathbf{D} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \\ \mathbf{D}^+ \end{bmatrix} \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \begin{bmatrix} \mathbf{D}^- \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \\ \mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \end{bmatrix}$$

The upper half of $\mathbf{D}$, $\mathbf{D}^-$, produces the derivatives on the negative x -axis, and the lower half of $\mathbf{D}$, $\mathbf{D}^+$, produces the derivatives on the positive x -axis.

Because of the symmetry properties of h and differentiation, we only need, in fact

$$\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} = \mathbf{D}^+ \mathbf{h}^\dagger$$

An interesting question, from the performance point of view, is whether we can compute this result without explicitly constructing

$$\mathbf{h}^\dagger = \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix}$$

which requires copying $\mathbf{h}$ twice.

We have, with $i = 0, \dots, 2N - 1$:

$$\left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i = \sum_{l=0}^{2N-1} D_{il}^\pm h_l^\dagger$$

$$= \sum_{l=0}^{N-1} D_{il}^\pm h_l^\dagger + \sum_{l=N}^{2N-1} D_{il}^\pm h_{l-N}^\dagger$$

where the first sum acts on $\pm \mathbf{g}$ and the second sum acts on $\mathbf{h}$.

$$= \sum_{l=0}^{N-1} D_{il}^\pm h_{N-1-l}^\dagger + \sum_{l=N}^{2N-1} D_{il}^\pm h_{l-N}^\dagger$$

$$= \pm \sum_{l=0}^{N-1} D_{il}^\pm h_{N-1-l}^\dagger + \sum_{l=0}^{N-1} D_{i,N+l}^\pm h_l^\dagger$$

$$= \pm \sum_{l=0}^{N-1} D_{il}^\pm h_{N-1-l}^\dagger + \sum_{l=0}^{N-1} D_{i,N+l}^\pm h_l^\dagger$$

$$= \pm \sum_{l=0}^{N-1} D_{il}^\pm h_{N-1-l}^\dagger + \sum_{l=0}^{N-1} D_{i,N+l}^\pm h_l^\dagger$$

$$\text{If we divide } \mathbf{D} \text{ in } 4 \times N \times N \text{ quadrants: } \mathbf{D} = \begin{bmatrix} \mathbf{D}^{\text{ul}} & \mathbf{D}^{\text{ur}} \\ \mathbf{D}^{\text{ll}} & \mathbf{D}^{\text{lr}} \end{bmatrix}$$

$$\left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i = \pm \sum_{l=0}^{N-1} D_{il}^{\text{ll}} h_{N-1-l} + \sum_{l=0}^{N-1} D_{i,l}^{\text{lr}} h_l$$

now with $i = 0, \dots, N - 1$. The second sum can be implemented using `numpy.einsum`, as in the non-reduced case. For the first sum, this is perhaps not possible because the access to is reversed. We can eliminate the reversed access to h , by reversing the access to D_{ij}^{ll} :
[eq 19]

$$\begin{aligned} \left[\mathbf{D}^+ \begin{bmatrix} \pm \mathbf{g} \\ \mathbf{h} \end{bmatrix} \right]_i &= \pm \sum_{l=0}^{N-1} D_{il}^{\text{ll}} h_{N-1-l} + \sum_{l=0}^{N-1} D_{i,l}^{\text{lr}} h_l \\ &= \pm \sum_{l=0}^{N-1} D_{i,N-1-l}^{\text{ll}} h_l + \sum_{l=0}^{N-1} D_{i,l}^{\text{lr}} h_l \\ &= \sum_{l=0}^{N-1} [\pm D_{i,N-1-l}^{\text{ll}} + D_{i,l}^{\text{lr}}] h_l \end{aligned}$$

$$= \sum_{l=0}^{N-1} [\pm E_{il}^{ll} + D_{il}^{lr}] h_l = \sum_{l=0}^{N-1} [D_{il}^{lr} \pm E_{il}^{ll}] h_l$$

Thus, by reversing the order of the columns in D_{ij}^{ll} we can avoid explicit construction of $\mathbf{h}^\dagger$, and allow for a `numpy.einsum` implementation. This is a one time cost, incurred when the $\mathbf{D}$ matrices are constructed. The $\pm$ is a $+$ when h is symmetric and a $-$ when h is skew-symmetric.

Higher order $\mathbf{D}$ matrices are constructed by multiplying them, e.g.:

$$\mathbf{D}^2 = \mathbf{D}^1 \mathbf{D}^1$$

Thus, the superscript indicating the order of the differentiation can be interpreted as a power. This implies that constructing the higher order $\mathbf{E}^{ll}$ and $\mathbf{D}^{lr}$ matrices have to be constructed from the full higher order $\mathbf{D}^n$ matrices.

Warning

I am not quite sure this is correct... This approach seems to ignore the effect of $\pm$ in eq 19. On the other hand, the derivation of eq 19 seems independent of whether $\mathbf{D}^+$ corresponds to $\mathbf{D}^1$, $\mathbf{D}^2$, ...

Testing will reveal the truth.

It probably is correct after all.

Likewise,

$$\begin{aligned} \frac{d^2 h}{dx^2} &= \frac{d}{dx} \left(\sum_{l=0}^{N-1} [D_{il}^{lr} \pm E_{il}^{ll}] h_l \right) \\ &= \sum_{k=0}^{N-1} [D_{ik}^{lr} \pm E_{ik}^{ll}] \sum_{l=0}^{N-1} [D_{kl}^{lr} \pm E_{kl}^{ll}] h_l \\ &= (\mathbf{D} \mp \mathbf{E})(\mathbf{D} \pm \mathbf{E})\mathbf{h} \end{aligned}$$

Here, the sign of $\mathbf{E}$, $\mp$, in the first factor is the opposite of $\mathbf{E}$ in the second factor because differentiation flips the symmetry (symmetric becomes skew-symmetric and v.v.). Thus if h is symmetric (skew-symmetric) the formula reads

$$\begin{aligned} \frac{d^2 \mathbf{h}}{dx^2} &= (\mathbf{D} - \mathbf{E})(\mathbf{D} + \mathbf{E})\mathbf{h}_{\text{symmetric}} \\ \frac{d^2 \mathbf{h}}{dx^2} &= (\mathbf{D} + \mathbf{E})(\mathbf{D} - \mathbf{E})\mathbf{h}_{\text{skew-symmetric}} \end{aligned}$$

Note

There is an additional catch when applying the $\mathbf{D} \pm \mathbf{E}$ formulas, namely that they depend on the symmetry of $\mathbf{h}$. As the different components of an observable may have different symmetries, we must apply it to each component separately. This complicates

the implementation, and may impact the performance. However, the memory use is 1/4th of the full matrix ($N \times N$ vs $2N \times 2N$). As this part of the code will probably be memory-bound, the reduced approach is most probably faster.

Note

Keeping track of the symmetry is non-trivial. It is perhaps better to first implement the full matrix approach for reduced axes and then use that to validate the approach based on eq 19. On the other hand, completing h_{ijk} to a non-reduced grid isn't easy either. After all keeping track of the symmetry turned out to be easy, provided the derivatives are computed as

[Eq 22]

$$[\mathbf{F}_x^{n_x}][\mathbf{F}_y^{n_y}][\mathbf{F}_z^{n_z}]\mathbf{h}$$

(i.e. by grouping the derivatives wrt the same coordinate axis in a single matrix multiplication), where $\mathbf{F} = \mathbf{D} \pm \mathbf{E}$ with $\pm$ selected from the symmetry behavior of $\mathbf{h}$ for the corresponding coordinate axis. This is because each matrix-multiplication can only change the symmetry behavior of its outcome with respect to the axis of the differentiation, not with respect to the other axes. That is, the symmetry behavior of the z -component of $[\mathbf{F}_z^{n_z}]\mathbf{h}$ may have changed, depending of the differentiation order n_z , but its symmetry behavior with respect to the x - and y -axis are unchanged. Consequentially, we can rely on the symmetry behavior of $\mathbf{h}$ for each of the multiplications.

Eq 22 presents also an optimal scheme for reusing previously computed derivatives. By incrementing the differentiation order by one and storing $[\mathbf{F}_y^{n_y}]\mathbf{h}$, $[\mathbf{F}_z^{n_z}]\mathbf{h}$ and $[\mathbf{F}_x^{n_x}][\mathbf{F}_z^{n_z}]\mathbf{h}$ every new derivative can be computed with a single matrix multiplication (because derivatives with respect to the same coordinate axis can be grouped in a single matrix multiplication). E.g to compute the Hessian, the following steps are taken:

$$\frac{d\mathbf{h}}{dx} = [\mathbf{F}_x^1]\mathbf{h}$$

$$\frac{d\mathbf{h}}{dy} = [\mathbf{F}_y^1]\mathbf{h}$$

$$\frac{d\mathbf{h}}{dz} = [\mathbf{F}_z^1]\mathbf{h}$$

$$\frac{d^2\mathbf{h}}{dx^2} = [\mathbf{F}_x^2]\mathbf{h}$$

$$\frac{d^2\mathbf{h}}{dy^2} = [\mathbf{F}_y^2]\mathbf{h}$$

$$\frac{d^2\mathbf{h}}{dz^2} = [\mathbf{F}_z^2]\mathbf{h}$$

$$\frac{d^2 \mathbf{h}}{dx dy} = [\mathbf{F}_x^{-1}] [\mathbf{F}_y^{-1}] \mathbf{h} = [\mathbf{F}_x^{-1}] \frac{d \mathbf{h}}{dy}$$

$$\frac{d^2 \mathbf{h}}{dx dz} = [\mathbf{F}_x^{-1}] [\mathbf{F}_z^{-1}] \mathbf{h} = [\mathbf{F}_x^{-1}] \frac{d \mathbf{h}}{dz}$$

$$\frac{d^2 \mathbf{h}}{dy dz} = [\mathbf{F}_y^{-1}] [\mathbf{F}_z^{-1}] \mathbf{h} = [\mathbf{F}_y^{-1}] \frac{d \mathbf{h}}{dz}$$

Every line requires only a single matrix multiplication, namely, the leftmost one as the operand to its right is either $\mathbf{h}$ or a or has been computed before. Furthermore, computing

$$\frac{d^2 \mathbf{h}}{dx^2} = [\mathbf{F}_x^{-2}] \mathbf{h}$$

from scratch has the same cost as

$$\frac{d^2 \mathbf{h}}{dx^2} = [\mathbf{F}_x^{-1}] \frac{d \mathbf{h}}{dx}$$

Note

Rather than storing $\mathbf{D}$ and $\mathbf{E}$, it is more efficient to store $\mathbf{D} + \mathbf{E}$ and $\mathbf{D} - \mathbf{E}$, where $\mathbf{D} = \mathbf{D}^{lr}$ and $\mathbf{E} = \mathbf{E}^{ll}$ are derived from the corresponding full higher order $\mathbf{D}$ matrix. So, for every differentiation order n , a full $\mathbf{D}^{(n)}$ matrix is stored for non-reduced coordinate axes, and for reduced coordinates axes two 1/4 matrices, $[\mathbf{D}^{(n)} + \mathbf{E}^{(n)}]$ and $[\mathbf{D}^{(n)} - \mathbf{E}^{(n)}]$, are stored.

N -dimensional grids

The full Cartesian 3D representation of a function $h(\mathbf{r})$, where $\mathbf{r} = [x \ y \ z]$ (3D case), is then provided (for the 3D case) by [eq 23]

$$h(\mathbf{r}) = \sum_{ijk} h_{ijk} f_i(x) f_j(y) f_k(z)$$

where the number of discretization points does not have to be the same in each direction.

Note that f_i , f_j and f_k are generally different objects, even if accidentally the indices i , j and k are identical, as they pertain, resp., to the x -axis, the y -axis and the z -axis, and the grid spacing is not necessarily the same.

Derivatives

In this case, the derivative matrices $\mathbf{D}^{(1)}$ and $\mathbf{D}^{(2)}$ have to be set up separately for each direction, taking into account whether the axis is reduced or not.

[eq 23.1]

$$\left. \frac{d\Phi}{dx} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ljk}$$

$$\left. \frac{d\Phi}{dy} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ilk}$$

$$\left. \frac{d\Phi}{dz} \right|_{ijk} = \sum_l D_{il}^1 \Phi_{ijl}$$

The summation is over the index of Φ_{ijk} that corresponds to the axis wrt which the derivative is taken: the derivative wrt $x/y/z$ sums over the 1st/2nd/3rd index. The same holds for 2nd, 3rd, 4th order derivatives ($\frac{d^n}{du^n}$, $u = x, y, z$, $n \geq 0$).

Note

`numpy.einsum` is ideally suited to compute sums like these.

Basis functions

The basis functions are plane wave products of the different axes:

[eq 24]

$$\begin{aligned} \Phi_{klm}(x, y, z) &= \phi_k(x) \phi_l(y) \phi_m(z) = \frac{1}{\sqrt{L_x}} \frac{1}{\sqrt{L_y}} \frac{1}{\sqrt{L_z}} \exp\left(\frac{2\pi j}{L_x} kx\right) \exp\left(\frac{2\pi j}{L_y} ly\right) \exp\left(\frac{2\pi j}{L_z} mz\right) \\ &= \frac{1}{\sqrt{L_x L_y L_z}} \exp\left(2\pi j \left(\frac{kx}{L_x} + \frac{ly}{L_y} + \frac{mz}{L_z}\right)\right) \\ k &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_x - 1}{2} \\ l &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_y - 1}{2} \\ m &= \pm \frac{1}{2}, \pm \frac{3}{2}, \dots, \pm \frac{N_z - 1}{2} \\ &= \frac{1}{\sqrt{L_x L_y L_z}} \exp(2\pi j (\mathbf{k} \cdot \mathbf{r})) \\ \mathbf{k} &= \begin{bmatrix} \frac{k}{L_x} & \frac{l}{L_y} & \frac{m}{L_z} \end{bmatrix} \\ \mathbf{x} &= [x \quad y \quad z] \end{aligned}$$

Note that ϕ_k , ϕ_l and ϕ_m are generally different objects, even if accidentally the indices k , l and m are identical, as they pertain, resp., to the x -axis, the y -axis and the z -axis.

Alternatively, in the spirit of eq 5.1:

[eq 24.1]

$$\mathbf{k} = \left[\frac{x_i}{L_x \Delta_x} \quad \frac{y_j}{L_y \Delta_y} \quad \frac{z_k}{L_z \Delta_z} \right]$$

where the x_i , y_j , z_k are grid coordinates in the x , y and z directions.

Note

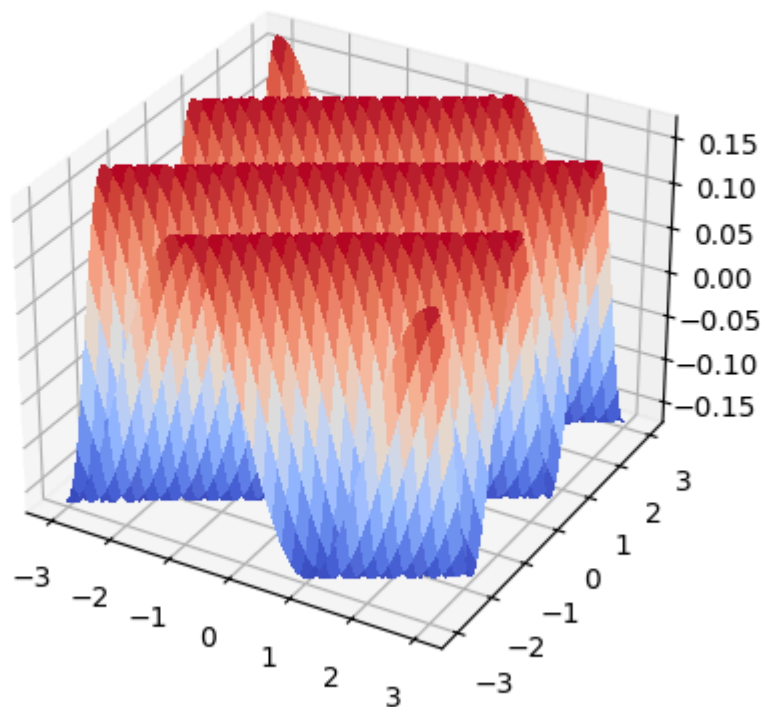
Contrary to the 1D case, it is generally not true for the 2D and 3D cases that the basis functions are symmetric or skew-symmetric.

E.g. changing the sign of the x -coordinate in $\exp(2\pi j(\mathbf{k} \cdot \mathbf{r})) = \exp(2\pi j(\mathbf{k} \cdot [-x, y, z]))$ yields

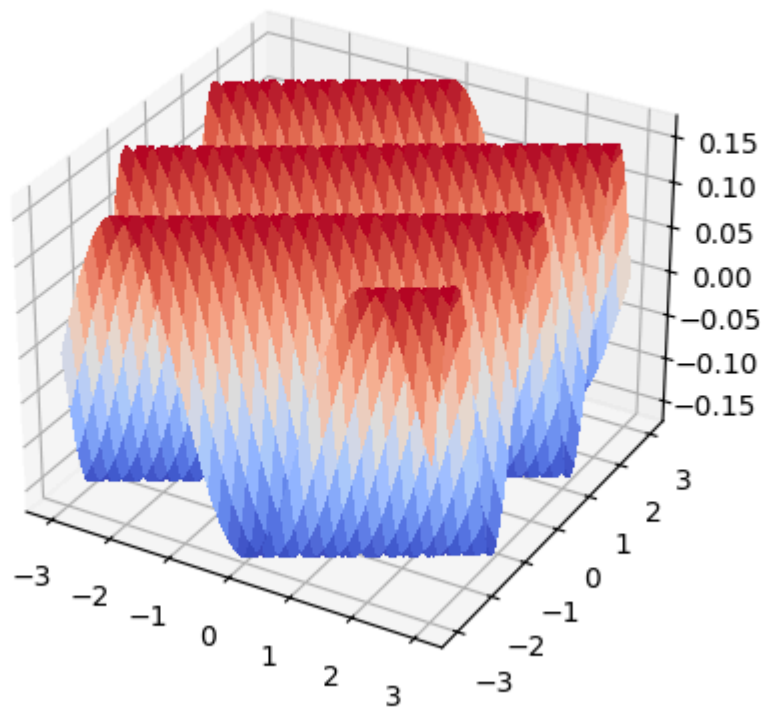
$$\cos(-k_x x + k_y y + k_z z) + j \sin(-k_x x + k_y y + k_z z)$$

and $\cos(-k_x x + k_y y + k_z z) = \cos(k_x x + k_y y + k_z z)$ only if $k_y y + k_z z$ is a multiple of 2π , which is generally not true. This can also be seen in the figures below. The plane waves are periodic only in the direction of the wave vector.

i=1, x_i=-1.5, y_j=2.5 real



i=1, x_i=-1.5, y_j=2.5 imag



Interpolation

As described by eq 23 Interpolating a scalar quantity h at a single point requires a sum over all grid points which may be costly (speaking of working interactively). If h needs to be interpolated on a large number of points, p ,

$$\mathbf{r} = \begin{bmatrix} x_0 & y_0 & z_0 \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_{p-1} & y_{p-1} & z_{p-1} \end{bmatrix}$$

it will be advantageous to move the loop over the points $0..p-1$ inside the loop over ijk product $h_{ijk}f_i(x)f_j(y)f_k(z)$.

In case h is not a scalar quantity but a tensor it is advantageous to move the loop over the components between the loop over the p interpolation points and the loop over ijk :

```
for all grid points ijk:
    for all components h of H
        for all interpolation points 0..p
            accumulate h_ijk * f_i(x_p) * f_j(y_p) * f_k(z_p) over ijk
```

The inner loop can be evaluated as a function over a numpy array:

```
for all grid points ijk:
    for all components h of H
        h(r) += h_ijk * f_i(r[:,0]) * f_j(r[:,1]) * f_k(r[:,2])
```

Furthermore, in case e.g. the x axis is reduced, according to eq 15 one must replace $f_i(x)$ by $(f_i(x) \pm f_{-i}(x))$, where the sign is $+$ if h is symmetric w.r.t. x and $-$ if h is skew-symmetric w.r.t. x .